

Thread-safe printf Debugging (Part 1)

by Nathan Jones

Table of Contents

Introduction.....	1
What could possibly go wrong?.....	1
“You shouldn’t use printf in an ISR anyways!”.....	3
A thread-safe toolbox.....	4
Disabling interrupts.....	4
Using a mutex.....	4
Reduce blocking time with a buffer.....	6
What could possibly go wrong? (Part Deux).....	7
Conclusion.....	13
Solutions.....	13
Races in a double buffer.....	13

Introduction

Debugging via `printf` (or `Serial.print` or `snprintf`) can be an extremely useful tool for quickly seeing what your system is doing and to zero in on the parts of your system that are, or could soon be, causing errors. This tool is not without its downsides, however, and we can potentially run into trouble if we use certain naive implementations from inside ISRs (interrupt service routines) or between RTOS threads. In this article we'll discuss what problems could potentially arise in these use cases and then do our best to make `printf/snprintf` **thread-safe** for use in ISRs and RTOS threads.

We'll start with a rather klunky solution and then improve on that. The updated solution will come with it's own problems, which we'll address in [Part 2](#).

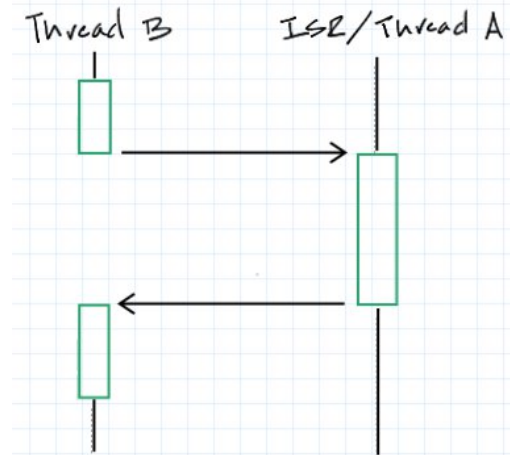
NOTE: We are not going to address multi-core processors in this article. I'm going to assume that all code shown is running on a single-core processor only.

What could possibly go wrong?

In [each of the last six articles](#) about `printf` debugging, we've been implicitly assuming that our system is single-threaded with no pre-emption: no ISRs (interrupt service routines) and no RTOS threads that could disrupt our carefully constructed messaging system. In a system that *does* use ISRs or a pre-emptive RTOS, however, it becomes entirely possible that a part of our system which is sending a message could get interrupted by another part of the system (that possibly also wants to send out a message!).

```
// Thread B
printf("Message 1");

// ISR/Thread A
printf("Message 2");
```

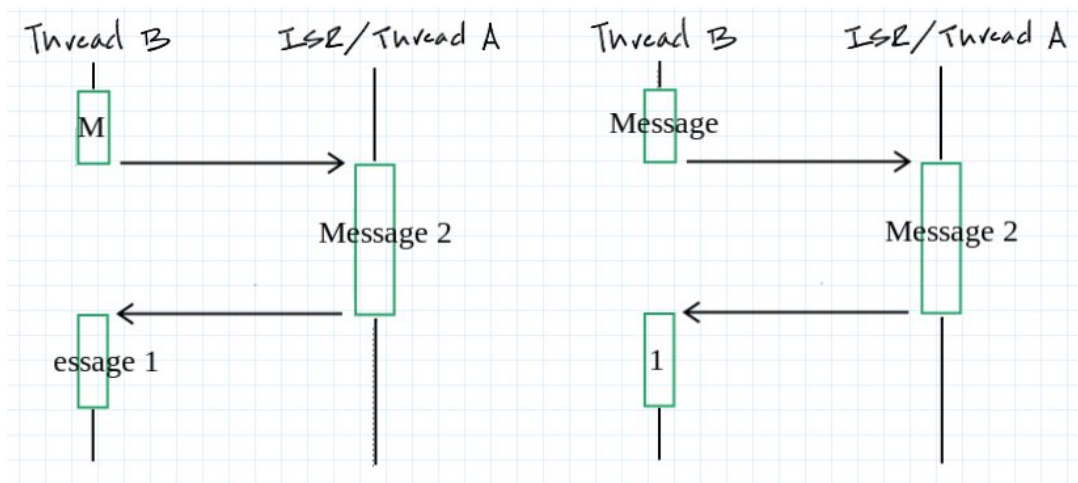


What's the worst that could happen in that scenario?



<https://imgflip.com/i/af8k9i>

At a *minimum*, our messages could get garbled! If “Thread B” was trying to send “Message 1” and the ISR/Thread A was trying to send “Message 2”, the output could be anything from “MMessage 2essage 1” (i.e. “Message 2” interrupted “Message 1” right after the first “M”) to “Message Message 21” (i.e. “Message 2” interrupted “Message 1” right before the last “1”).



This is known as a **race condition**, since the output of our program depends on a “race” between the two threads; programs with race conditions are, by definition, *not* **thread-safe**. In this scenario, the race occurred because our use of `printf` didn’t **protect concurrent accesses to shared resources**, i.e. to the UART or USB peripheral registers. Threads that use the peripheral are allowed to interleave, which is how we could possibly get garbled messages. We’ll discuss how to fix this after a brief digression about whether you should use `printf` in an ISR in the first place.

“You shouldn’t use `printf` in an ISR anyways!”

Conventional wisdom says that ISRs should be exceedingly short, long enough to set a variable or copy a byte from one place to another, roughly, and not much longer. Even using a fast serial communication channel, `printf` could take much longer to complete than those handful of instructions so, per this conventional wisdom, we should avoid using it in an ISR. And while this piece of wisdom isn’t *wrong*, per se, it over-simplifies things.

The reason ISRs “should” be short is because they hog your processor; while an interrupt is being serviced, the processor can’t handle other interrupts or perform any other task; they are blocked. (On systems with the ability to nest interrupts, higher-priority interrupts would still be able to be run, so only the main loop and interrupts with the same or lower priority as the current interrupt would be blocked.) Whether or not that’s a bad thing for your system depends on how much blocking time you can afford to have. If you have a 1 ms timer interrupt that keeps track of system time for you, then your other ISRs probably shouldn’t take longer than 1 ms. But if the ISR with `printf/post` is *truly* the highest priority task in your system and if the other tasks in your system *can* afford to wait the time it takes to make that call to `printf`, then you shouldn’t feel bad about putting that function call in your ISR, regardless of how long it takes.

For a more thorough discussion of determining whether a system can afford the blocking time of a call to `printf` inside an ISR, see [“You Don’t Need an RTOS”](#) on [EmbeddedRelated.com](#).

A thread-safe toolbox

To solve the problem of our garbled output, our system needs to prevent ISRs and threads from accessing the UART/USB peripheral while it's in use by another thread. We can do this straightforwardly in one of two ways:

- Disabling interrupts before accessing the peripheral
- Protecting the peripheral with a mutex

In fact, these two solutions are what we'll use, in general, to make our code thread-safe. Both, unfortunately, come with downsides, which we'll discuss in a brief introduction to each technique. I feel compelled to state, though, that **I don't recommend using the solutions below in practice, at least not without the modification** that I'll introduce in [the following section](#).

Disabling interrupts

For example:

```
disableISR();  
printf(...);  
enableISR();
```

By disabling interrupts before calling `printf` we can completely eliminate the possibility that another ISR or higher-priority thread could run before our message is done being sent.

This solution is clean and easy, but also *very* heavy-handed. It has the downside of preventing *all* interrupts on our system from running while a message is being sent, which might make other tasks in our system miss their deadlines. For the same reason above that we don't necessarily want our ISRs to be long (see [“You shouldn't use printf in an ISR anyways!”](#)), we also don't want to turn them off for very long, either.

Using a mutex

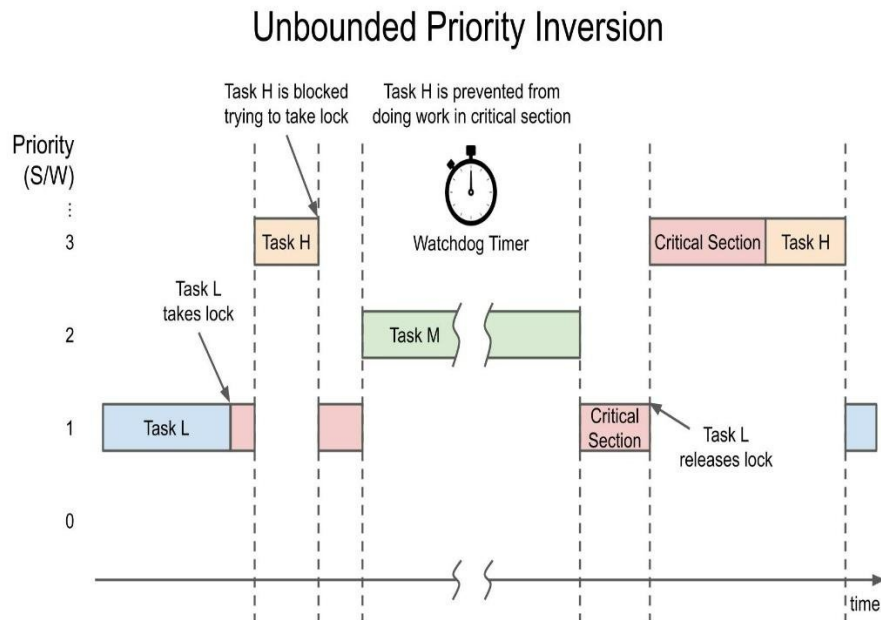
For example:

```
mutex_t printfMutex;  
if( lock(printfMutex) )  
{  
    printf(...);  
    unlock(printfMutex);  
}
```

Instead of turning off all interrupts, we can use a mutex (mutual exclusion lock) to lock down just `printf` while it's being used. As soon as a function acquires the mutex and begins executing `printf`, it prevents any other function from entering the `if` block until the mutex is unlocked.

Threads can execute this code in a loop to block until their message gets added to the send buffer. ISRs should never block, though, so they should only execute this code once and hope that they can add their message to the buffer.

This illustrates that although using a mutex is more surgical than disabling all interrupts, it now means that some messages *may* get skipped entirely. Additionally, we've opened up the possibility for a **priority inversion**, which is what would happen if a lower priority thread were to block a higher priority thread.



<https://media.geeksforgeeks.org/wp-content/uploads/20250901151313582147/unboundedpriorityinversionn.jpeg>

Think it couldn't happen? The sequence of events goes like this:

- A low-priority thread acquires a mutex
- A high-priority thread pre-empts the low-priority thread and tries to take the same mutex. It can't, so it yields back to the low-priority thread.
- A medium-priority thread pre-empts the low-priority thread and does whatever the heck it wants, for as long as it wants. The high-priority thread is effectively being blocked by a medium-priority thread!

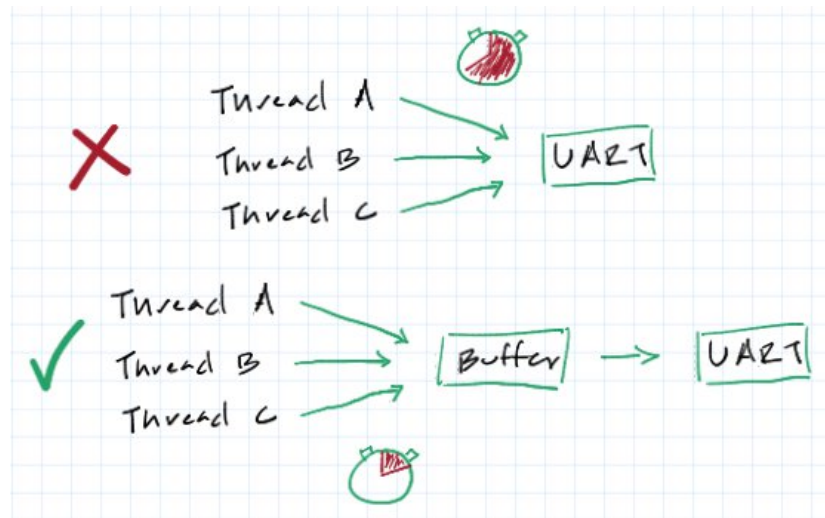
Some RTOSes have ways of handling this (temporarily increasing the priority level of a task that's holding a resource that a higher-priority task also wants to use, called "priority inheritance"), but, to me, it's like fixing your Bolivian tree lizard problem with a Chinese needle snake problem: you haven't really fixed anything, you've just given yourself a different problem.



https://y.yarn.co/fdb0b908-d844-477e-9ef5-2ba4ad437d20_text.gif

Reduce blocking time with a buffer

The solutions above have the downside of locking out part or all of our system while `printf` is executing, which could be quite a long time. We can alleviate much of this blocking time by posting messages to a **buffer** instead of directly to the UART/USB peripheral.



As long as the code that copies data from the buffer to the UART peripheral runs in its own thread (likely the UART “transmit buffer empty” ISR) and no other code tries to copy data directly to the UART data register, then we’ve successfully synchronized access to the UART peripheral (i.e. we’ve made it thread-safe). All we would need to do is synchronize access to the *buffer*. We’ll use our same tools as above (disabling interrupts or adding a mutex) but in this case their blocking time is much reduced, since copying an *N* byte message into a buffer will *always* run in less time than transmitting that same *N* byte message over a serial channel.

Here’s a sample buffer that we can use for this purpose. This is a single buffer, so immediately after a message is copied to the buffer, the UART ISR is enabled which will send out one byte of the buffer at a time until it’s empty. While that’s happening, no other message can be posted to the buffer. Once the buffer is empty, the UART ISR disables itself (until the next message comes in).

Listing 1: A simple, 256-byte buffer

```

#define MAX_MSG_LEN 256
char buffer[MAX_MSG_LEN+1] = {0};
volatile size_t nextIdx = 0, msgLen = 0;

size_t post(char * data, size_t len)
{
    // TODO: Handle data == NULL, len == 0
    size_t copied = 0;
    if(msgLen == 0) // Alt: if(!UART_TXE_IS_ENABLED())
    {
        msgLen = (len > MAX_MSG_LEN) ? MAX_MSG_LEN : len;
        memcpy(buffer, data, msgLen);
        nextIdx = 0;
        copied = msgLen;
        UART_TXE_ENABLE();
    }
    return copied;
}

void UART_TXE(void)
{
    UART_DR = buffer[nextIdx++];
    if(nextIdx == msgLen)
    {
        msgLen = 0;
        UART_TXE_DISABLE();
    }
}

```

Access to the UART peripheral is now thread-safe, since the only thread that uses it is the UART_TXE ISR (provided no other part of our program is also trying to use the UART peripheral). But how do we protect the buffer itself? To answer that (i.e. to find the right places to put our ISR disables or our mutexes), we need to ask, “What race conditions exist in this code?”

What could possibly go wrong? (Part Deux)

Wet white dog. Freshly mowed lawn.
What could possibly go wrong?



<https://i.imgur.com/AxzUprg.jpeg>

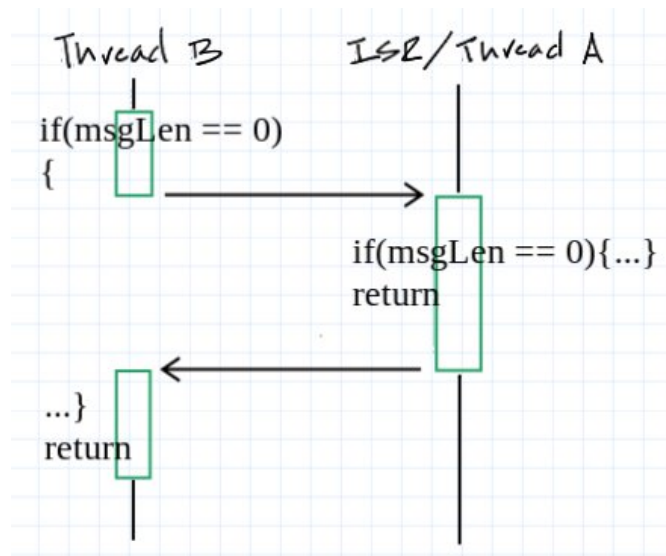
To identify our race conditions, we need to look out for a few things in our code:

1. Code that accesses shared resources, such as hardware peripherals (or our buffer!)
2. Code that is intended to be “atomic”, i.e. “indivisible”; in other words, a section of code that, once begun, *must* be finished before another thread should be allowed to enter the same section
 1. The section of code that mustn’t be interrupted is called a **critical section**.
3. Code that can’t happen until *after* a specific event
4. Code that has to happen *immediately after* a specific event

Looking at Listing 1, what races do you see?

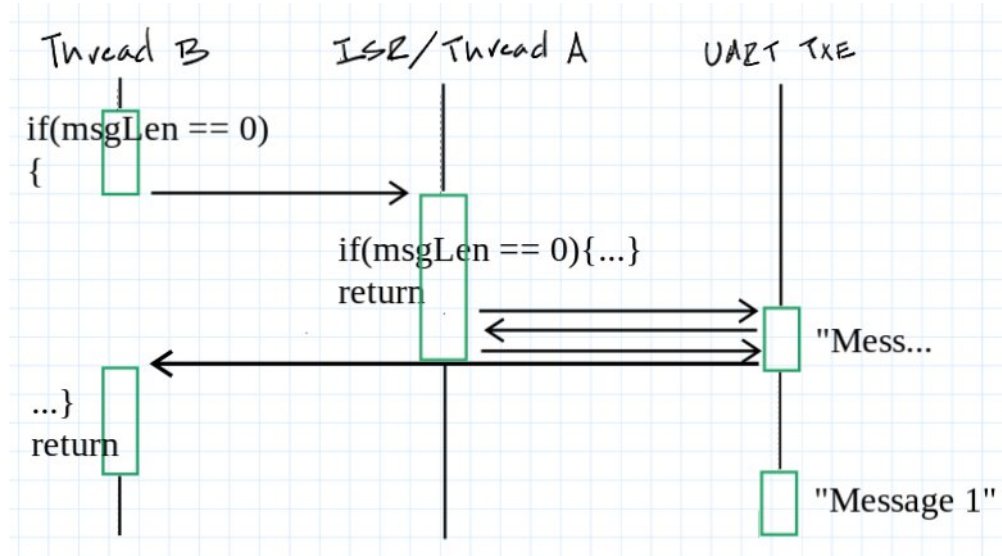
Did you identify that the `if(msgLen == 0)` block in `post()` is supposed to be atomic (#2, on our list above)? Once one thread has entered this block and begun to put a message into buffer, *no other thread* should be allowed in that block until the message has been fully transmitted. More specifically, it’s the first two lines (the conditional check, `if(msgLen == 0)`, and the next line, `msgLen = (len > MAX_MSG_LEN) ? MAX_MSG_LEN : len`) that need to happen atomically; once `msgLen` takes on a non-zero value, the `if` statement will keep other threads out by design.

This race condition looks like:

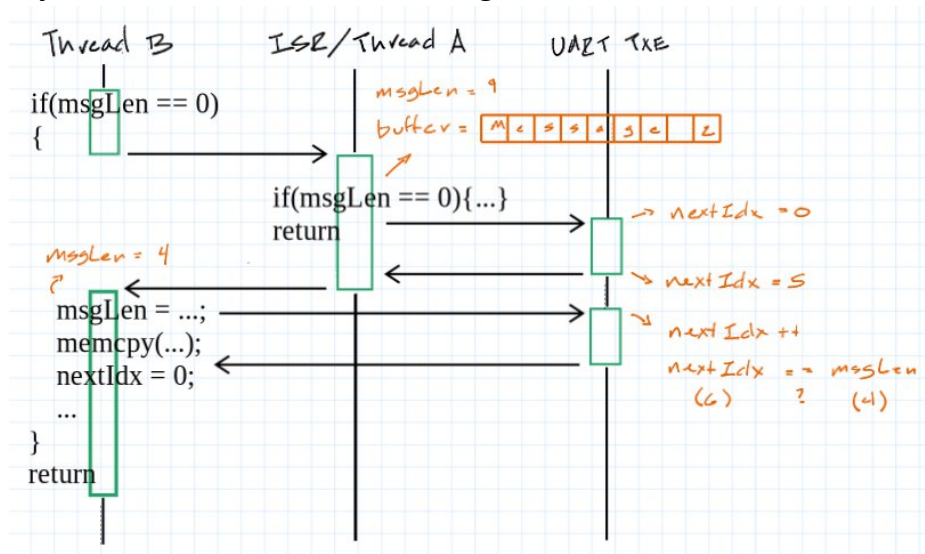


1. Thread B sees an empty buffer and enters the `if(msgLen == 0){...}` block.
2. Before Thread B updates `msgLen`, Thread A/ISR runs, also posting a message into the buffer.
3. Thread B is returned to. The exact error to happen next depends on several factors, but could be:
 1. The UART ISR is able to run several times before Thread B copies its data into the buffer and resets `msgLen` and `nextIdx`. We see the first few characters of Thread A’s message

and then the entirety of Thread B's message.

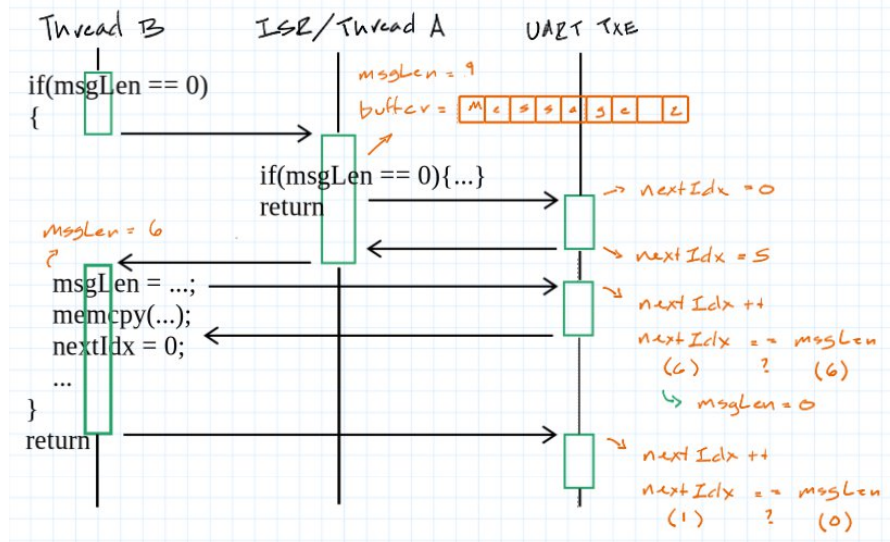


- The UART ISR is able to run N times before Thread B is returned to (so `nextIdx == N`). Thread B sets `msgLen` to a value smaller than N, meaning `nextIdx == msgLen` will never be true by only incrementing `nextIdx` (at least, until `nextIdx` wraps around). The ISR runs several more times before Thread B sets `nextIdx` to 0, possibly printing values beyond the end of Thread A's message.



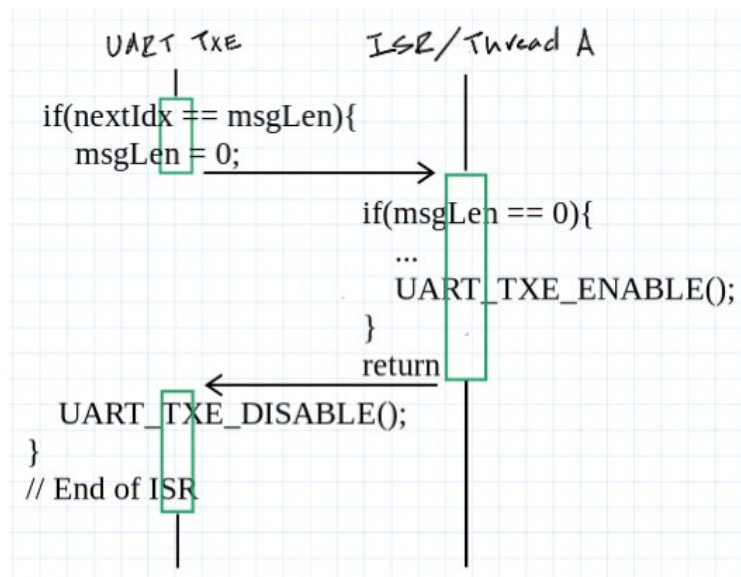
- The UART ISR is able to run N times before Thread B is returned to (so `nextIdx == N`). Thread B sets `msgLen` to N+1 such that the next time the UART ISR runs it thinks the message is done being sent (`nextIdx == msgLen`). The ISR then sets `msgLen` to 0 and disables itself. Thread B continues copying to the buffer, resetting `nextIdx` to 0. When the UART ISR is re-enabled, `nextIdx` is incremented to 1, which doesn't equal `msgLen` (which is 0), resulting in printing values beyond Thread B's message until `nextIdx` wraps

around or the processor encounters an error trying to read from unreadable memory.

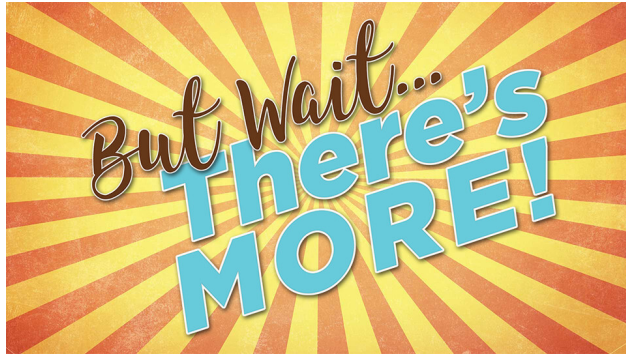


Did you also identify that the `if(nextIdx == msgLen)` block in `UART_TXE()` is also supposed to be **atomic**? Once the UART ISR has determined that it's done sending a message, setting `msgLen` to 0 and disabling itself are supposed to happen simultaneously.

This race condition looks like:



1. UART_TXE runs, determines that the message it's sending is done, and enters the `if(nextIdx == msgLen)` block. It sets `msgLen` to 0.
2. At that exact moment, a higher-priority ISR runs that wants to send out a message. It sees that `msgLen` is 0, so it copies its contents into the buffer and enables the UART_TXE ISR.
3. The first instance of UART_TXE is returned to, which *disables the UART_TXE ISR*. No further messages can be sent out because, improbably, `msgLen` is non-zero but the ISR has been disabled.



https://s3.amazonaws.com/assets.reformedjournal.com/wp-content/uploads/2024/11/05222009/But_Wait_web.jpg

Did you identify that `UART_TXE_ENABLE ( )` can't run until after `msgLen`, `buffer`, and `nextIdx` have been updated (#3, on our list above)? Thanks to either optimizing compilers or out-of-order processors, though, there's a chance that our code *doesn't even execute in the order in which it's written*. The compiler/processor is allowed to do this so long as any re-orderings preserve the intent of the program, which it can do in a lot of cases that still violate what we expect to happen. For instance, here's what the central `if` block in `post ( )` looks like from the perspective of the compiler/processor:

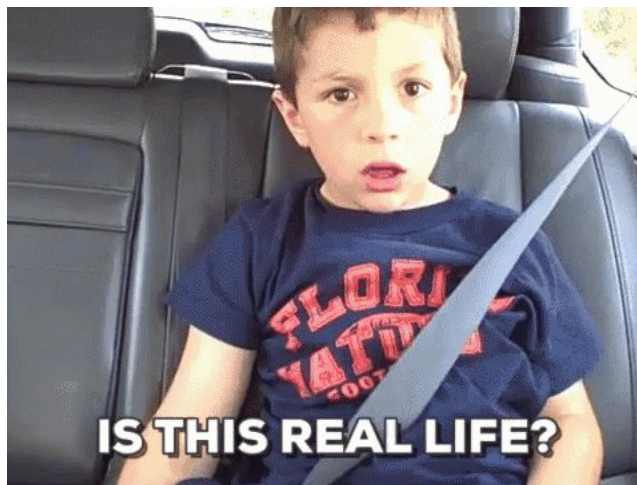
```
01 w = (R1 > #) ? # : R1;
02 memcpy(x, R0, w);
03 y = 0;
04 R0 = w;
05 z = 1;
```

There are a few data dependencies here (lines 2 line 4 have to come after line 1, owing to the dependence on `w` [i.e. `msgLen`], for instance), but otherwise it doesn't seem like we could do any harm by moving the other stuff around, right?

```
01 w = (R1 > #) ? # : R1;
02 y = 0;
03 z = 1;
04 memcpy(x, R0, w);
05 R0 = w;
```

The compiler/processor is allowed to do this for performance reasons. Perhaps, by keeping certain operations together, it can keep a variable in a register longer. Maybe it's on a cache line that's about to get ejected. And we do like our speed, right??

Do you see the problem, though? `z` is our stand-in for the UART ISR register; if `UART_TXE_ENABLE ( )` (i.e. `z = 1;`) runs before `memcpy`, the ISR will immediately trigger (the transmit register *is* empty, after all), but the data won't have been set!



https://media.tenor.com/W_PpFowDI3YAAAAe/is-this-real-life-shock.png

I'm afraid so, David. In fact, we don't even know at this point if this is even all of the race conditions we need to worry about! There's not actually any formal way to *prove* that a function or a program is free from race conditions, which also means that there are no automated tools to check for us, either. The closest we can get, AFAIK, are tools that can check for **data races**, which was item #1 on our list above (e.g. RV-Predict, ThreadSanitizer, Intel Inspector, CodeSonar, etc).

Challenge question!

We were using a double buffer in the three “Faster printf” articles, not a single buffer. The code to post a message to that double buffer was:

```
#define BUFFER_LEN 512
char buffer[2][BUFFER_LEN] = {0};
size_t using = 0;
char * next = (char*)buffer;
size_t room = BUFFER_LEN;

size_t post(char * data, size_t len)
{
    if(len > room)
    {
        // Send filled buffer
        while(USB_OK != CDC_Transmit_FS(...));

        // Swap (update status variables)
        using = 1 - using;
        next = (char*)&buffer[using];
        room = BUFFER_LEN;
    }

    // Copy msg to current buffer
    memcpy(next, msg, len);
    next += len;
    room -= len;

    return len;
}

uint8_t CDC_Transmit_FS(uint8_t* Buf, uint16_t Len)
```

```
{
    // If not transmitting: copy Buf/Len to internal buffer
    // and wait for IN token packet from Host
}
```

Can you find all of the potential race conditions in that code? (Solution [here](#).)

Conclusion

Doing any sort of serial debugging (`printf`, `sprintf`, `mpack`, `FlatBuggers`, `bit-proto`, etc) requires specific mechanisms for thread-safety in an ISR or RTOS environment. This is true if, for no other reason, than we want to ensure that messages can't be inadvertently intermixed while they're being transmitted.

The easiest way to protect access to the hardware peripheral are to:

1. Disabling interrupts before accessing the peripheral
2. Protecting the peripheral with a mutex

These two techniques are the beginning of our “thread-safe toolbox”. The downside to using these techniques around calls to `printf`, though, is that they may block (some or all of) the rest of our application while a message is being transmitted. A better solution, one that vastly decreases our blocking time, is to copy messages into a buffer before being sent out the UART/USB port. We'll still need to block while a message is being copied, but the time to copy a message to another spot in memory will inevitably be vastly shorter than waiting for the serial peripheral.

A single buffer, shown in Listing 1, is about as simple a buffer as we can get. Even still, there were several race conditions lurking in those 17 lines of code. We can identify race conditions by analyzing our code for

1. Code that accesses shared data, such as hardware peripherals
2. Code that is intended to be “atomic”, i.e. “indivisible”; in other words, a section of code that, once begun, *must* be finished before another thread should be allowed to enter the same section
3. Code that can't happen until after a specific event
4. Code that has to happen immediately after a specific event

What can we do to eliminate those new race conditions?? We'll be using our “thread-safe toolbox” for sure, but I'll save the details until [Part 2](#). =D

If you've made it this far, thanks for reading and happy hacking!

Solutions

Races in a double buffer

The first ones that jump out at me are the two places where we have a set of variables that all need to be updated together:

```
// Swap (update status variables)
using = 1 - using;
next = (char*)&buffers[using];
room = BUFFER_LEN;
```

and

```
// Copy msg to current buffer
memcpy(next, msg, len);
next += len;
room -= len;
```

These are both critical sections and will need to be protected from simultaneous access by two or more threads.

In fact, the entire `if (len > room){...}` block is a critical section, since we wouldn't want one thread to try swapping the buffers at the same time *another* thread was trying to do the same.

And, now that I'm looking at it, I think the *entire function* is a critical section, just because of how I've designed it, since a low priority thread might think `len <= room` (i.e. there's room in one of the buffers) only to have that buffer filled up before the low priority thread is able to complete its `memcpy`.

So, at face value, we'd have to protect the entire function to be thread-safe:

```
size_t post(char * data, size_t len)
{
    // TODO: Handle data == NULL, len == 0, > 512
    uint32_t primask_state = __get_PRIMASK();
    __disable_irq();
    if (len > room)
    {
        // Send filled buffer
        // Fail instead of spin?
        while (USB_D_OK != CDC_Transmit_FS(...));

        // Swap (update status variables)
        using = 1 - using;
        next = (char*)&buffers[using];
        room = BUFFER_LEN;
    }

    // Copy msg to current buffer
    memcpy(next, msg, len);
    next += len;
    room -= len;

    __set_PRIMASK(primask_state);
    return len;
}
```

However, I think this is a sub-optimal design (in terms of thread-safety) and that it's possible to re-write it slightly so that not *all* of the function needs to be inside the critical section.

Challenge question!

(What?! A challenge question *inside* a challenge question??)

How could you redesign the double buffer so that the critical section didn't need to be as long (i.e. so you could move certain long operations like `CDC_Transmit_FS` or `memcpy` outside the critical section)? (No solution this time; this is an open-ended question!)